

Nicolas De Brebisson

Charlie Merchadou

Gaëtan Billiard

Rapport du projet POTE

Groupe POTIMARRON

Projet N° INFO_03 : Katamino

Sommaire

Introduction.....	3
Présentation	3
Objectif.....	3
I. Implémentation du Katamino	4
A. Implémentation des formes.....	4
B. Implémentation de l'espace et du Jeu	5
II. Algorithme de résolution.....	6
A. Principe et fonctionnement.....	6
Figure 4 : Arbre de recherche de l'algorithme de resolution du Katamino.	7
B. Première étude de vitesse de résolution sans optimisations	7
1. La fonction render().....	9
2. La fonction en elle-même.....	10
III. Optimisations.....	11
A. Eviter les espaces vides de taille inférieure à 5	11
B. Contraindre le placement sur les cases idéales.....	12
C. Étude de la vitesse d'exécution et autres statistiques.....	13
Conclusion :	14
Bibliographie :	15
Annexe A : Capture d'écran des fonctions importantes de notre code avec des commentaires et des explications.....	16
Annexe B : Toutes les combinaisons de pièces possibles pour remplir une grille en fonction de sa dimension.....	23

Introduction

Dans le cadre de l'UE POTE 4, le groupe de projet POTIMARRON, composé de Nicolas DE BREBISSON, Charlie MERCHADOU et Gaëtan BILLIARD, a été chargé de réaliser le projet n° INFO_03, intitulé *Katamino*, sous la responsabilité de M. Stéphane BONNEVAY.

Présentation

Katamino est un casse-tête bidimensionnel créé le 1er janvier 1992 par André Perriolat. Le but du jeu est de placer l'ensemble des pièces dans une grille de dimensions variables allant de 3×5 à 12×5 , sans laisser d'espace vide.

Les pièces utilisées dans ce jeu sont des figures géométriques constituées de cinq carrés accolés par l'un de leurs côtés. Ces formes sont appelées des *pentominos* (dont la liste est présentée ci-dessous).

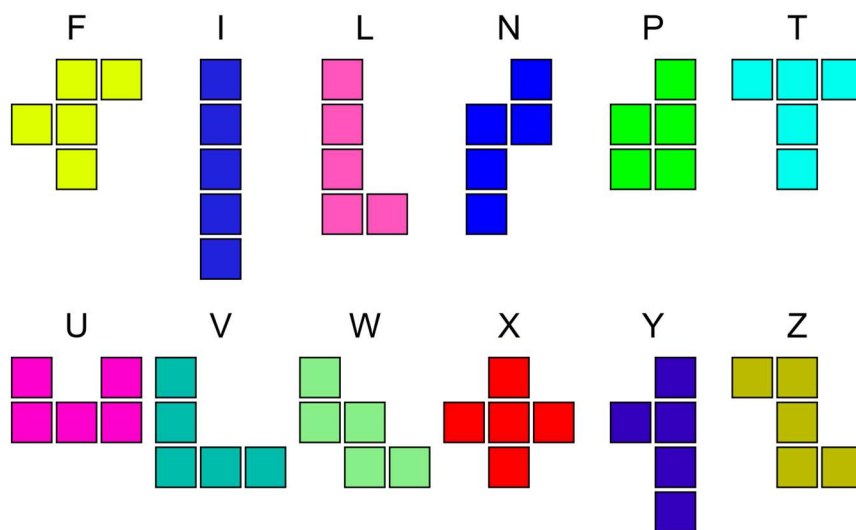


Figure 1 : Liste des Pentomino

Objectif

L'objectif de ce projet est de concevoir un algorithme en C++ capable de générer, afficher graphiquement et résoudre automatiquement le Katamino à partir d'une liste de pièces et des dimensions de l'espace de jeu données.

Le projet vise également à étudier et à implémenter différentes méthodes permettant d'optimiser la résolution du puzzle.

I. Implémentation du Katamino

Dans un premier temps, nous allons nous pencher sur l'implémentation du Katamino en C++, notamment sur la construction des formes, de l'espace de jeu, des fonctions indispensables au fonctionnement du jeu et son affichage graphique.

A. Implémentation des formes

Tout d'abord, nous avons dû coder l'élément indispensable du jeu : les pentominos (également appelés *pentaminos* de manière informelle).

Pour cela, nous avons commencé par développer une classe générale nommée Shape, définissant les propriétés et les fonctions communes à chacune des formes du jeu. Ensuite, grâce à un lien d'hérédité, nous avons défini chacun des 12 pentominos utilisés dans le jeu.

Figure 2.a : classe principale Shape

```
class Shape{
protected: //attributs communs à chaque forme
    int rotateState; //Donne l'état de rotation de la pièce
    int cubeSize=20; //taille du cube pour l'affichage graphique
    int x,y; //Coordonnées de la forme
    bool flipped; //indique si la pièce est retournée ou non
public:
    char id;
    Cell shape[5]; //Tableau contenant les 5 carrés constituant la forme
    //Toute les fonctions indispensables au bon fonctionnement du code en C++
    //et permettant de tourner, retourner, bouger et de récupérer l'état de la pièce
    Shape();
    ~Shape();
    int getRota();
    void render();
    void rotate();
    virtual void move();
    void flip();
    bool getFlip();
    void operator =(Shape s2);
    bool operator !=(Shape s2);
};
```

Figure 2.b : 2 classes formes

```
//Exemples de classes de formes.
//X X
//XXX
class U: public Shape{//done
public:
    U();
    ~U();
    void move() override;
};

//XXX
//XX
class P: public Shape{//fini
public:
    P();
    ~P();
    void move() override;
};
```

Pour l'affichage graphique du Katamino, nous avons choisi d'utiliser la bibliothèque Raylib. Dans les classes définissant les formes, les fonctions *move()* et *render()* exploitent cette bibliothèque afin d'assurer l'affichage et la manipulation graphique des pièces.

La fonction *render()* permet d’afficher la pièce en dessinant individuellement chacun des carrés composant la forme générée, tandis que la fonction *move()* permet de manipuler la pièce à l’aide de la souris.

Grâce à cette implémentation, nous sommes capables de créer, déplacer, faire pivoter, retourner et afficher chacune des formes du Katamino.

B. Implémentation de l’espace et du Jeu

Après avoir implémenté les différentes pièces, il était nécessaire de concevoir la grille de jeu.

Pour cela, nous avons créé une nouvelle classe nommée *Tableau*, permettant de générer la grille et de manipuler les différentes formes.

```
class Tableau{
private:
    Cell tab[5][12]; //La grille
    int nbligne;//Nombre x de ligne du tableau de dimension x*5
    Shape* shapes[12]; //tableau contenant toutes les formes
    Shape availShape[63]; //tableau contenant les formes et chacune de leurs rotations possibles
    int nbPossibilities;
    SavePlacement placedShapes[12]; //toutes les pièces placées
    int nbPlacedShapes; //le nombre de formes placées, permet de manipuler les tableaux
    saveState notAllowed[12][63*12*5]; //tableau contenant les formes et chacune de leurs
    //rotations possibles qui ont déjà été placées
    int nbNotAllowed[12]; //Le nombre de pièces qui ont été placé, permet de manipuler les tableaux
}
```

Figure 3.a : attributs de la classe tableau avec explications

```
public:
    Tableau(int nbl=12); //Met de manière aleatoire les formes dans le tableau availShapes
    ~Tableau();
    void render(); //Affichage avec raylib
    bool canPlace(int indices, int x, int y); //Vérifie que l'on peut placer notre forme
    //Permet de ne pas avoir de sortie de grille ou de pièces qui se chevauchent
    int nbOpti(int indices, int x, int y); //renvoie le nombre de cases occupées à côté de l'endroit où l'on veut placer notre forme
    void placeShape(int indices, int x, int y); //Permet de poser notre forme et modifie le tableau shapes
    //Le prochain élément à placer doit être le premier element du tableau (shapes[0])
    bool solve();
    void removeShape(); //Enlève de la grille la dernière pièce du tableau placedShapes et désincrémente nbPlacedShapes.
```

Figure 3.b : fonctions de la classe tableau avec explications

Une nouvelle fois, la fonction *render()* permet l’affichage de la grille à l’aide de la bibliothèque Raylib.

Ainsi, cette nouvelle classe permet non seulement de générer la grille de jeu, mais également de manipuler les pentominos, de les placer dans la grille et de les en retirer.

Maintenant que l’ensemble de la structure du jeu a été implémenté, nous pouvons passer à l’étape suivante : concevoir un algorithme capable de

résoudre n'importe quel Katamino en fonction de la grille et de la liste de pièces fournies.

II. Algorithme de résolution

A. Principe et fonctionnement

Afin de résoudre automatiquement le jeu, nous avons utilisé un algorithme de *backtracking*.

Un algorithme de retour sur trace, également appelé retour arrière (*backtracking* en anglais), désigne une famille d'algorithmes permettant de résoudre des problèmes algorithmiques, notamment des problèmes de satisfaction de contraintes. Ces algorithmes consistent à tester systématiquement l'ensemble des affectations possibles afin de trouver une solution valide.

Lorsqu'aucune solution ne peut être trouvée à partir d'une configuration donnée, l'algorithme abandonne cette branche de recherche et revient aux affectations précédentes pour explorer d'autres possibilités. Ce mécanisme explique l'appellation de « retour sur trace » ou « retour arrière ». Ce type d'algorithme est notamment utilisé pour résoudre des casse-têtes tels que le Sudoku.

Dans notre cas, l'algorithme doit être capable de remplir entièrement la grille de jeu en y plaçant les différentes pièces. La condition d'arrêt est atteinte lorsque toutes les pièces ont été placées avec succès et que la grille est complètement remplie.

Le retour sur trace intervient lorsque l'algorithme ne parvient plus à placer de nouvelle pièce alors que la grille n'est pas entièrement complétée. Dans ce cas, il revient à l'étape précédente en retirant la dernière pièce placée, puis poursuit les tests avec une autre configuration possible.

La fonction de *backtracking* est présentée en annexe A, accompagnée d'explications détaillées.

Afin de rendre ce type d'algorithme plus compréhensible, il est courant d'utiliser des arbres de recherche permettant de schématiser le

fonctionnement du procédé de *backtracking*. Ces arbres sont construits de la manière suivante :

- La racine de l'arbre correspond au point de départ de l'algorithme, c'est-à-dire à la solution vide.
- Les enfants d'un nœud sont obtenus en étendant la solution, c'est-à-dire en répertoriant toutes les configurations possibles à partir de ce nœud. Ainsi, chaque enfant représente une solution potentielle.

Dans notre cas, l'arbre de recherche associé à notre algorithme de retour sur trace est présenté ci-dessous.

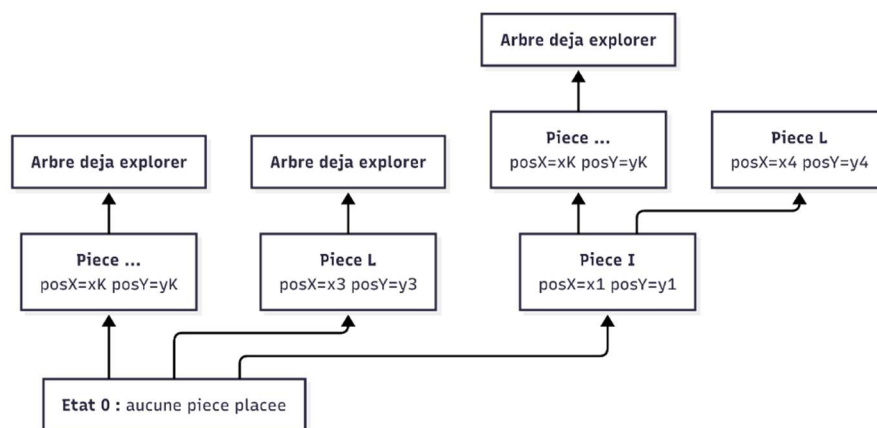


Figure 4 : Arbre de recherche de l'algorithme de resolution du Katamino

B. Première étude de vitesse de résolution sans optimisations

Nous avons réalisé une première étude de la vitesse de résolution avec cette version de notre programme, c'est-à-dire sans optimisation particulière, à l'exception de la recherche de placement optimal. Cette étude a été menée sur des grilles de dimensions 5×3 , 5×4 , 5×5 , 5×6 , 5×7 et 5×8

Pour cela, nous avons procédé de la manière suivante :

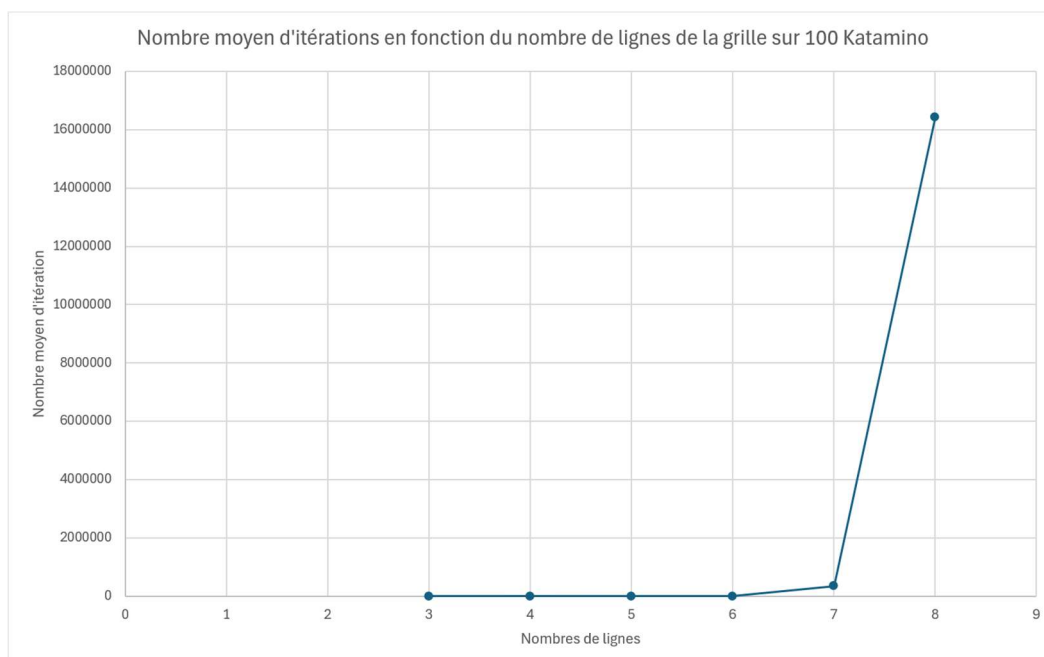
- On mesure le nombre d'itérations nécessaire ainsi que le temps d'exécution requis pour résoudre un Katamino
- On répète les mesures 100 fois, l'étude portant donc sur 100 Kataminos de même dimension
- On calcule la moyenne sur 100 Kataminos des différentes mesures obtenues.

Afin de simplifier l'étude et d'automatiser le processus, nous avons développé un programme dédié. Celui-ci lance successivement la résolution des 100 Kataminos et récupère, pour chacun d'eux, le nombre d'itérations ainsi que le temps nécessaire à sa résolution.

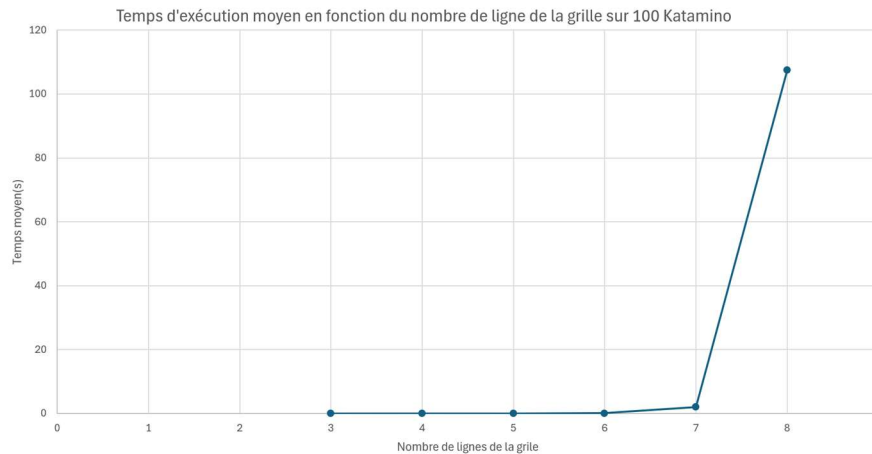
Enfin, le programme calcule la moyenne des deux mesures sur l'ensemble des 100 exécutions, puis affiche les résultats dans la console afin de pouvoir les exploiter.

nombre de lignes	itération moyenne	temps moyen (en s)
3	38	0,0001824
4	18	0,000125199
5	394	0,0019026
6	5571	0,0285133
7	354948	2,00793
8	16423075	107,527

Tab 1 : Données d'étude pour la fonction non optimisée



Graphe 1 : Étude du nombre moyen d'itération pour résoudre un Katamino sur 100 instances



Graph 2 : Étude du temps d'exécution moyen pour résoudre un Katamino sur 100 instances

À la suite de cette étude, nous observons, à partir des graphiques, une croissance combinatoire explosive, potentiellement exponentielle ou pire, du temps d'exécution moyen en fonction du nombre de lignes de la grille. Le tableau met également en évidence l'augmentation très importante du nombre d'itérations nécessaires, notamment pour une grille de taille 5×8 , où le nombre d'itérations atteint huit chiffres.

C'est pour cela que l'étude sur des grilles de dimensions supérieures n'a pas été réalisée, car le temps nécessaire à leur exécution aurait été beaucoup trop important, pouvant atteindre plusieurs dizaines d'heures, voire une journée entière, pour les raisons décrites dans la suite de cette partie.

Au cours de cette étude, nous avons identifié plusieurs facteurs de ralentissement dans notre implémentation :

1. La fonction `render()`

L'un des éléments ralentissant la vitesse d'exécution du programme est la fonction `render()`.

Pour rappel, cette fonction utilise d'autres fonctions de la bibliothèque Raylib afin d'afficher graphiquement le Katamino.

Cependant, l'affichage graphique peut être coûteux en temps d'exécution, en fonction des performances de la machine utilisée, et peut fausser les mesures de temps réalisées lors des différentes études de vitesse d'exécution. C'est pour cela que toutes les mesures de performance sont effectuées sans affichage graphique afin d'obtenir des résultats fiables.

Si l'on souhaite néanmoins conserver un affichage graphique, il est possible de n'effectuer le rendu du Katamino qu'après un certain nombre d'opérations réalisées par le programme.

```
375         if (iter%50000 == 0){ //Affichage toute les 50 000 itérations
376             render();
377         }
```

Figure 5 : Condition d'affichage

2. La fonction en elle-même

De manière générale, notre fonction met beaucoup de temps à résoudre le Katamino. Par exemple, voici le nombre d'exécution nécessaire pour résoudre un Katamino avec une grille de 5×12 à l'aide de ce code non optimisé, dans le cas le plus favorable :

```
done in 18869055 iteration
|
```

Figure 6 : itérations pour un Katamino de 5×12 (18 869 055)

Dans d'autres cas, le programme peut nécessiter beaucoup plus de temps pour résoudre un Katamino de 5×12 , avec un nombre d'itérations pouvant atteindre neuf, dix, voire onze chiffres si l'on attend suffisamment pour qu'il termine la résolution du puzzle.

Cela est notamment dû au fait que la fonction peut placer des pièces et laisser des espaces vides trop petits pour permettre l'ajout d'une pièce, comme l'illustre la capture d'écran ci-dessous, les cases grises représentant des cases vides :

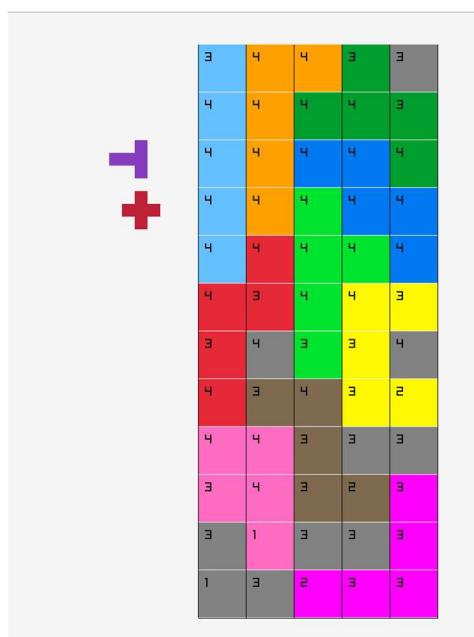


Figure 7 : illustration du cas où le programme laisse des espaces non comblables

Si les pièces sont placées et qu'une case vide est laissée dès le début de la résolution, le code mettra beaucoup de temps avant de revenir sur ce placement problématique. De plus, nous avons déjà observé qu'après être revenu sur ce choix de placement, le programme pouvait reproduire la même erreur avec une autre pièce et doit donc revenir une nouvelle fois sur le placement de celle-ci.

Ainsi, pour résoudre ce problème, nous pourrions introduire une nouvelle fonction qui « forcerait » le code à ne laisser aucun espace vide de taille inférieure à 5.

De ce fait, l'étude de vitesse d'exécution n'a pu être réalisée que sur des grilles de 5×3 , 5×4 , 5×5 , 5×6 , 5×7 et 5×8 . Au-delà de ces dimensions, le temps d'exécution augmente fortement en raison de l'explosion combinatoire inhérente au backtracking. Par exemple, pour une grille de 5×9 , le temps de résolution est d'environ deux heures.

III. Optimisations

Afin d'améliorer la vitesse de résolution du Katamino par notre programme, nous avons mis en place différentes optimisations.

A. Eviter les espaces vides de taille inférieure à 5

Dans un premier temps, nous avons développé une fonction nommée `hasIsolatedRegion()` (fonction commentée et détaillée en annexe A), permettant d'éviter que le programme ne laisse des espaces vides trop petits pour accueillir une pièce, c'est-à-dire des zones de taille inférieure à 5. Cette optimisation vise à résoudre le problème décrit dans la partie II.B.2 du rapport.

Concrètement, à partir de la pièce posée, cette fonction part de chaque case de la pièce pour lister les cases connectées vides. La fonction s'arrête après avoir parcouru 5 cases vides et renvoie une erreur lorsque l'espace disponible était insuffisant pour accueillir un pentomino.

On peut voir immédiatement émerger un autre cas d'erreur : si un espace vide n'est pas de taille divisible par 5, il ne peut pas exister de collection de pièces de taille 5 qui le remplissent. On peut alors modifier notre fonction

pour ne pas s'arrêter après avoir parcouru 5 cases vides et changer le test final.

```
if (optiMaxInd == -1){
    //cas où le programme n'arrive pas à placer de pièce
    if(nbPlacedShapes == 0){
        return -1;
    }
    removeShape();
}else{
    placeShape(optiMaxInd, optiMaxX, optiMaxY ); //on place notre pièce
    if (hasIsolatedRegion(optiMaxInd, optiMaxX, optiMaxY )){
        removeShape(); // on retire la case qu'on vient de mettre si elle génère des vides pas comblable
    }
}
```

Figure 8 : L'ensemble des cas d'erreur de notre fonction

B. Contraindre le placement sur les cases idéales

Dans un second temps, nous avons développé une fonction permettant de déterminer la case la plus contrainte de la grille, c'est-à-dire celle pour laquelle le nombre de placements possibles est le plus faible. Pour cela, nous utilisons la fonction *mostConstrained()* (commentée et détaillée en annexe A).

Cette fonction parcourt l'ensemble des cases encore libres de la grille, puis analyse toutes les pièces disponibles afin de déterminer le nombre de placements possibles à partir de chaque case. En se basant sur ces résultats, elle identifie la case la plus contrainte, c'est-à-dire celle à partir de laquelle la prochaine pièce devra être placée.

La fonction retourne true lorsqu'une case libre ne possède aucune possibilité de placement, et false dans le cas contraire. Cela ajoute ainsi un nouveau cas d'échec à notre algorithme.

Dans notre algorithme de backtracking optimisé, on appelle donc en premier *mostConstrained()* afin de déterminer la case à partir de laquelle poser notre pièce. Voir image ci-dessous :

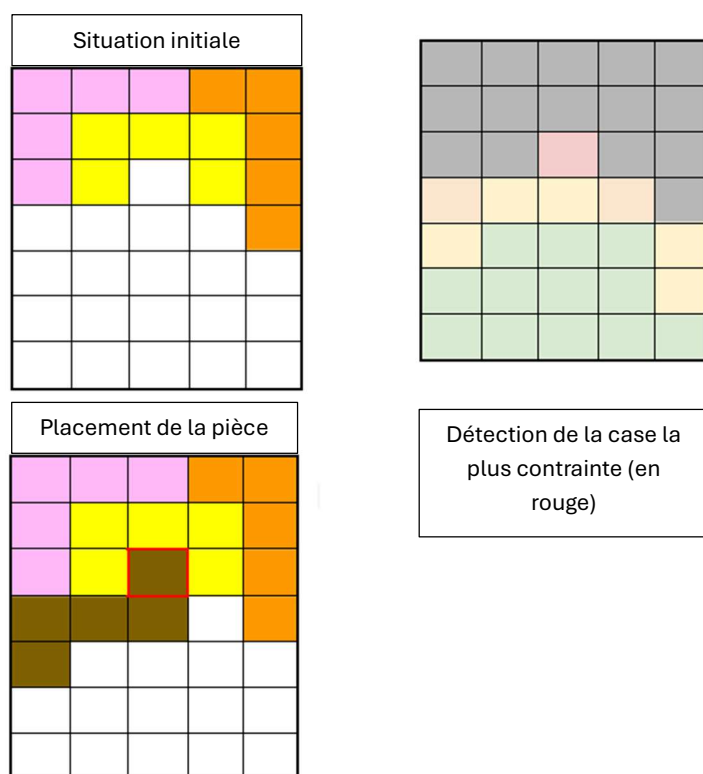


Figure 9 : illustration du fonctionnement de *mostConstrained()*

Cette fonction apporte une amélioration majeure de l'efficacité de l'algorithme, car elle oriente la recherche vers les branches les plus pertinentes de l'arbre de recherche (voir partie II.A, figure 4).

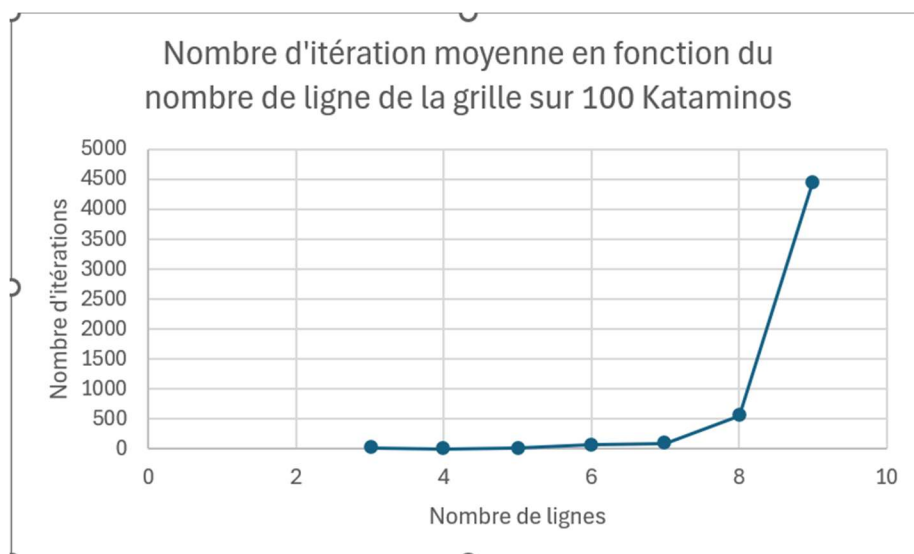
C. Étude de la vitesse d'exécution et autres statistiques

Nous avons ensuite réalisé une nouvelle étude des performances de notre programme après l'ajout des différentes optimisations.

Pour cela, nous avons appliqué le même protocole expérimental que précédemment : 100 Katamino générés à partir de listes de pièces aléatoires sont résolus automatiquement, puis les moyennes du temps d'exécution et du nombre d'itérations nécessaires à la résolution sont calculées sur l'ensemble des 100 exécutions.

Nombre de lignes	itération moyenne
3	19
4	5
5	16
6	71
7	92
8	563
9	4440

Tableau 2 : Etude de vitesse avec la fonction optimisée



Graph 3 : Étude du nombre moyen d'itération pour résoudre un Katamino sur 100 Kataminos

Tout d'abord, à partir des données du tableau 2, nous constatons que les optimisations apportées à notre programme améliorent considérablement la vitesse moyenne de résolution des puzzles. Par exemple, pour une grille de taille 5×8 , l'ancienne version du programme nécessitait près de 16 millions d'itérations pour résoudre un Katamino, tandis que cette nouvelle version n'en requiert plus que 563, soit une réduction d'un facteur proche de 10 000.

De plus, le graphe 3 met en évidence de manière plus claire le comportement de la croissance combinatoire de notre algorithme de retour sur trace. Bien que le nombre d'itérations continue d'augmenter avec la taille de la grille, cette croissance reste désormais beaucoup plus maîtrisée grâce aux optimisations mises en place.

Conclusion :

Ainsi, à travers ce projet, nous avons réussi à implémenter un environnement de jeu reproduisant le fonctionnement d'un Katamino physique (formes, grille de jeu, etc.), ainsi qu'un algorithme de retour sur trace capable de résoudre n'importe quel Katamino à partir d'une liste de pièces ordonnée aléatoirement et d'une grille de dimensions données.

Nous avons également étudié le comportement combinatoire de cet algorithme lors de la résolution, puis apporté plusieurs optimisations

permettant d'améliorer significativement les performances globales du programme.

Enfin, nous avons répertorié l'ensemble des combinaisons de pièces possibles pour différentes dimensions de grille (liste présentée en annexe B pour certains cas). Cette étude a notamment mis en évidence le rôle particulier de la pièce I qui, lorsqu'elle est placée correctement, revient à résoudre un Katamino de dimension inférieure.

Répartition du travail :

- Implémentation de l'environnement : Nicolas DE BREBISSON
- Fonction de Backtracking : Gaëtan BILLIARD
- Optimisations : Charlie MERCHADOU
- Rapport : Gaëtan BILLIARD
- Présentation : Tous

Bibliographie :

Figure 1 : <https://fr.wikipedia.org/wiki/Pentomino>

Histoire du Katamino : <https://www.jedisjeux.net/jeu-de-societe/katamino>

Figure 2 : Captures d'écran de notre code

Figure 3 : Captures d'écran de notre code

Backtracking: https://fr.wikipedia.org/wiki/Retour_sur_trace

Arbres de recherche pour retour sur trace :

https://fr.wikipedia.org/wiki/Retour_sur_trace

Figure 4 : Arbre de recherche créé par Nicolas DE BREBISSON

Figure 5 : Capture d'écran de notre code

Figure 6 : Capture d'écran de la console affichant le retour du programme

Figure 7 : Capture d'écran de l'affichage graphique

Figure 8 : Capture d'écran de notre code

Annexe A : Captures d'écran des fonctions importantes de notre code, accompagnées de commentaires et d'explications.

Toutes les images présentées ci-dessous sont des captures d'écran de notre code.

1. Fonction placeShape()

Cette fonction permet de placer une pièce dans la grille. `indicesS` désigne un indice du tableau `availShape` (qui contient toutes les pièces utilisables), tandis que `x` et `y` représentent les coordonnées de la case sur laquelle on souhaite placer le centre de la pièce (déterminé dans la déclaration de chaque pièce).

La fonction place ensuite la pièce dans un autre tableau, `placedShape`, qui regroupe toutes les pièces déjà placées et qu'il ne faut donc pas replacer.

```
void Tableau::placeShape(int indicesS, int x, int y){
    //Placement de la pièce
    for(int i=0; i<5; i++){
        int posX=x+availShape[indicesS].shape[i].posX;
        int posY=y+availShape[indicesS].shape[i].posY;
        tab[posX][posY].color=availShape[indicesS].shape[i].color;
        tab[posX][posY].take=true;
        for(int j=-1; j<2; j++){
            for(int k=-1; k<2; k++){
                if(j*j!=k*k && posX+j>=0 && posX+j<5 && posY+k>=0 && posY+k<nbLigne){
                    tab[posX+j][posY+k].opti++;
                }
            }
        }
    }

    //mise de la pièce dans le tableau de pièce placé
    placedShapes[nbPlacedShapes].posX=x;
    placedShapes[nbPlacedShapes].posY=y;
    placedShapes[nbPlacedShapes].id=availShape[indicesS].id;
    placedShapes[nbPlacedShapes].indiceDansTab=indicesS;

    //on fait en sorte que le programme n'aille pas récupérer à nouveau cette pièce
    notAllowed[nbPlacedShapes][nbNotAllowed[nbPlacedShapes]].flip=availShape[indicesS].getFlip();
    notAllowed[nbPlacedShapes][nbNotAllowed[nbPlacedShapes]].rota=availShape[indicesS].getRota();
    notAllowed[nbPlacedShapes][nbNotAllowed[nbPlacedShapes]].id=availShape[indicesS].id;
    notAllowed[nbPlacedShapes][nbNotAllowed[nbPlacedShapes]].posX=x;
    notAllowed[nbPlacedShapes][nbNotAllowed[nbPlacedShapes]].posY=y;

    nbNotAllowed[nbPlacedShapes]++;
    nbPlacedShapes++;
    //on pose une forme donc on reset la prochaine
}
```


2. Fonction canPlace()

Cette fonction retourne un booléen indiquant si la pièce que l'on souhaite placer peut effectivement être ajoutée à la grille.

Si le placement est valide, la fonction retourne true. En revanche, elle retourne false dans les cas suivants :

- Certaines cases du placement sont déjà occupées par une autre pièce
- Le placement dépasse les limites de la grille
- La pièce que l'on souhaite placer est déjà présente dans la grille.

indices désigne un indice du tableau availShape (qui contient toutes les pièces utilisables), tandis que x et y représentent les coordonnées de la case sur laquelle on souhaite placer le centre de la pièce.

```
bool Tableau::canPlace(int indices, int x, int y){
    //Fonction vérifiant si notre pièce peut être placée en x y
    int posX, posY;
    for(int i=0; i<5; i++){
        posX=x+availShape[indices].shape[i].posX;
        posY=y+availShape[indices].shape[i].posY;
        if(posX<0 || posX>4 || posY<0 || posY>nbLigne-1) return false; //si la pièce sors de la grille
        if(tab[posX][posY].take) { //si la case ou l'une des cases qui serait occupé par notre pièce est déjà prise
            return false;
        }
    }

    for(int i=0; i<nbPlacedShapes; i++){ //vérifie si la pièce n'a pas déjà été placée
        if (placedShapes[i].id==availShape[indices].id) return false;
        if (placedShapes[i].indiceDansTab==indices) return false;
    }

    for(int j=0; j<nbNotAllowed[nbPlacedShapes]; j++){ //longue condition -->
        if(notAllowed[nbPlacedShapes][j].rota==availShape[indices].getRota() && notAllowed[nbPlacedShapes][j].id==av
        {return false;}
    }
    return true;
}
```

3. Fonction nbOpti()

Cette fonction calcule le score d'optimisation d'un placement de pièce, utilisé dans la recherche du placement optimal dans l'algorithme de backtracking.

indices désigne un indice du tableau availShape, tandis que x et y représentent les coordonnées de la case sur laquelle on souhaite placer le centre de la pièce.

```

int Tableau::nbOpti(int indices,int x,int y){
    //Fonction calculant le score d'optimisation d'un placement
    int nbOpti=0;
    for(int i=0;i<5;i++){
        nbOpti+=tab[x+availShape[indices].shape[i].posX][y+availShape[indices].shape[i].posY].opti;
    }
    return nbOpti;
}

```

4. Fonction removeShape()

Cette fonction retire la dernière pièce placée et supprime cette pièce du tableau des pièces déjà placées.

```

void Tableau::removeShape() {
    //Retirer une pièce
    int x, y, indices;
    nbNotAllowed[nbPlacedShapes] = 0;
    nbPlacedShapes--;
    //on récupère la pièce précédemment placée
    indices = placedShapes[nbPlacedShapes].indiceDansTab;
    y = placedShapes[nbPlacedShapes].posY;
    x = placedShapes[nbPlacedShapes].posX;
    //et on l'enlève de la grille (en diminuant l'opti des cases qui étaient adjacentes à la pièce enlevé)
    for(int i=0;i<5;i++){
        int posX=x+availShape[indices].shape[i].posX;
        int posY=y+availShape[indices].shape[i].posY;
        for(int j=-1;j<2;j++){
            for(int k=-1;k<2;k++){
                if(j*j!=k*k && posX+j>=0 && posX+j<5 && posY+k>=0 && posY+k<nbLigne){
                    tab[posX+j][posY+k].opti--;
                }
            }
        }
        tab[posX][posY].color=WHITE;
        tab[posX][posY].take=false;
    }
}

```

5. Fonction algorithmeDePlacage()

Notre fonction principale de backtracking.

Elle commence par rechercher le placement le plus optimisé ainsi que la pièce correspondante (recherche du maximum sur le score d'optimisation de chaque pièce), tout en vérifiant que le placement est possible.

Si une ou plusieurs solutions sont trouvées, le programme sélectionne le placement (pièce et coordonnées) possédant le score d'optimisation le plus élevé. Sinon, un retour arrière est effectué.

```

int Tableau::algorithmeDePlacage(){
    while (!WindowShouldClose()) //Affichage graphique jusqu'à ce qu'on ferme la fenêtre
        //DEBUT
        while(!WindowShouldClose() && (nbPlacedShapes != nbLigne)){ //Condition d'arrêt
            //std::cout<<nbPossibilities<<std::endl;
            iter ++; // nombre d'itérations nécessaires pour résoudre le Katamino (+1 à chaque passage de boucle)
            optiMax = 0; //Score de la meilleur pièce à placer
            optiMaxInd = -1; //indice correspondant à la meilleur pièce à placer dans availShape
            optiMaxX; //Coordonnées optimale pour la meilleur pièce à placer
            optiMaxY;
            opti;
            //On détermine la pièce et l'emplacement optimale pour le prochain placement
            //Pour cela on utilise plusieurs variables et une fonction capable de calculer
            //via un score si l'emplacement est optimal ou non (recherche de max)
            for (int i = 0; i<63; i++){
                for (int x = 0; x < 5; x++){
                    for (int y = 0; y < nbLigne; y++){
                        if (canPlace(i, x, y)){
                            if ((opti = nbOpti(i, x, y)) > optiMax){
                                optiMax = opti;
                                optiMaxInd = i;
                                optiMaxX = x;
                                optiMaxY = y;
                            }
                        }
                    }
                }
            }

            if (optiMaxInd == -1){
                //Cas d'erreur, le programme n'arrive à placer aucune pièce
                //On enlève donc la pièce précédente
                removeShape();
            }else{
                //Cas où le programme place une pièce
                placeShape(optiMaxInd, optiMaxX, optiMaxY );
            }

            if (iter%50000 == 0){
                //Un affichage toute les 50 000 itérations
                render();
            }
        }
    } // FIN
}

```

6. Fonction hasIsolatedRegion()

Cette fonction vérifie si, autour d'une pièce passée en paramètre, la grille comporte des espaces vides dont la taille n'est pas divisible par 5 et qui ne peuvent donc pas être remplis.

```

bool Tableau::hasIsolatedRegion(int indices, int x, int y) const {
    bool visited[5][nbLigne]; // tableau des cases déjà visitées (pour éviter trop de complexité)
    for (int i = 0; i < 5; i++){ // on ne regarde que les voisins directs de la pièce qu'on vient de poser
        int px = x + availShape[indices].shape[i].posX;
        int py = y + availShape[indices].shape[i].posY;

        int di[] = {-1,1,0,0}; //directions possibles en i et j
        int dj[] = {0,0,-1,1};
        for (int d = 0; d < 4; d++) {
            int ni = px + di[d];
            int nj = py + dj[d];
            // si le voisin est vide et pas encore testé
            if (ni>=0 && ni<5 && nj>=0 && nj<nbLigne && !tab[ni][nj].take && !visited[ni][nj]){
                // parcours en largeur graph depuis ce voisin (file)
                int queue[nbLigne*5][2];
                int head = 0, tail = 0;
                queue[tail][0] = ni; queue[tail][1] = nj; tail++;
                visited[ni][nj] = true;
                int count = 0;
                while (head < tail) {
                    int ci = queue[head][0], cj = queue[head][1]; head++;
                    count++;

                    for (int d2 = 0; d2 < 4; d2++){ //on cherche les voisins à enfiler de chaque case défilé
                        int mi = ci + di[d2];
                        int mj = cj + dj[d2];
                        if (mi>=0 && mi<5 && mj>=0 && mj<nbLigne && !tab[mi][mj].take && !visited[mi][mj]){
                            visited[mi][mj] = true;
                            queue[tail][0] = mi; queue[tail][1] = mj; tail++;
                        }
                    }
                }
                if (count % 5 != 0) return true; //si après avoir tous parcourus, l'espace n'est pas de taille modulo 5, true
            }
        }
    }
    return false; //sinon, si aucun cas est problématique, false
}

```

Fonctionnement :

Pour chacune des cinq cases de la pièce, une file est créée. Chaque case vide adjacente est ajoutée à cette file puis marquée comme déjà visitée.

Aussitôt, on défile la case précédente en incrémentant le compteur de taille de la région. Quand la file est vide, alors le compteur correspond à la taille de la région. Si cette taille n'est pas divisible par 5 alors la fonction s'arrête et renvoie "true" (= "il y a une région problématique").

Le même processus étant effectué pour chaque case de la pièce, le fait de garder en mémoire les cases déjà visitées est important pour éviter trop de calcul. On a très rarement plus de 2 espaces clos autour d'une même pièce.

Si on arrive à la fin de la fonction, c'est qu'il n'y a pas eu de cas problématique, la fonction retourne donc "false"

7. Fonction mostConstrained

Cette fonction détermine et retourne les coordonnées x et y passées en paramètres de la case la plus contrainte de la grille, ainsi qu'un booléen valant true lorsqu'il existe une case à partir de laquelle aucune pièce ne peut être placée.

```
bool Tableau::mostConstrained(int &bestX, int &bestY){ // on cherche la case la plus contrainte
    int minOption = 2147483647;
    bestX = -1; bestY = -1;

    for(int x=0; x<5; x++){ // parcours des cases du tableau
        for(int y=0; y<nbLigne; y++){

            if(tab[x][y].take) continue; // exclus les cases prises
            int option = 0;

            for(int i=0; i<nbPossibilities; i++){ // parcours de availShape
                for(int c=0; c<5; c++){ // parcours des cases d'une piece

                    int aX = x - availShape[i].shape[c].posX;
                    int aY = y - availShape[i].shape[c].posY;
                    if (canPlace(i, aX, aY)) option++;

                }
            }
            if(option==0) return true; // il existe une case sans possibilités => backtrack

            if(option < minOption){
                minOption = option;
                bestX = x;
                bestY = y;
            }
        }
    }
    return false; // il n'y a pas de case sans possibilité
}
```

Fonctionnement :

La fonction parcourt toutes les cases encore libres de la grille, puis analyse l'ensemble des pièces disponibles afin de déterminer le nombre de placements possibles à partir de chaque case (variable option).

Selon la valeur d'option :

- si option = 0 alors il existe une case sans placement possible : la fonction s'arrête et retourne "true".
- si option < minOption alors on garde en mémoire notre case comme la meilleure en mettant à jour minOption, bestX et bestY. On continue notre parcours de la grille

- sinon on continue notre parcours de la grille

À la fin de la fonction, false est retourné, ce qui signifie qu'aucun cas où option = 0 n'a été rencontré.

8. Fonction `algorithmeDePlacage()`

Version optimisée de notre fonction de *backtracking*, intégrant les différentes optimisations développées au cours du projet.

```
//DEBUT
while(!WindowShouldClose() && (nbPlacedShapes != nbLigne)){
    iter++;
    optiMax = 0;
    optiMaxInd = -1; //-1 = cas d'erreur, le programme n'a réussi à placer aucune des pièces dans availShape

    // on cherche la case la plus contrainte
    render();
    int bestX, bestY;
    if(mostConstrained(bestX, bestY) && nbPlacedShapes>0){
        removeShape(); // backtrack si une case n'a pas d'option
    }else {
        if(bestX!=-1 && bestY!=-1){
            for(int i=0; i<nbPossibilities; i++){ // parcours de availShape
                for(int c=0; c<5; c++){ // parcours des cases d'une piece

                    int aX = bestX - availShape[i].shape[c].posX;
                    int aY = bestY - availShape[i].shape[c].posY;

                    if (canPlace(i, aX, aY)){
                        int o = nbOpti(i, aX, aY);
                        if(o > optiMax){
                            optiMax = o;
                            optiMaxInd = i;
                            optiMaxX = aX;
                            optiMaxY = aY;
                        }
                    }
                }
            }
        }
        if (optiMaxInd == -1){
            //cas où le programme n'arrive pas à placer de pièce
            if(nbPlacedShapes == 0){
                return -1;
            }
            removeShape();
        }else{
            placeShape(optiMaxInd, optiMaxX, optiMaxY); //on place notre pièce
            if (hasIsolatedRegion(optiMaxInd, optiMaxX, optiMaxY)){
                removeShape(); // on retire la case qu'on vient de mettre si elle génère des vides pas comblable
            }
        }
    }
}
//FIN
```

Annexe B : Toutes les combinaisons de pièces possibles pour remplir une grille en fonction de sa dimension.

Nous avons aussi répertorié toutes les combinaisons possibles de pièces pour remplir une grille en fonction de sa taille de 5*3 à 5*12

Légende : Toutes les formes seront symbolisées par une lettre de la manière suivante

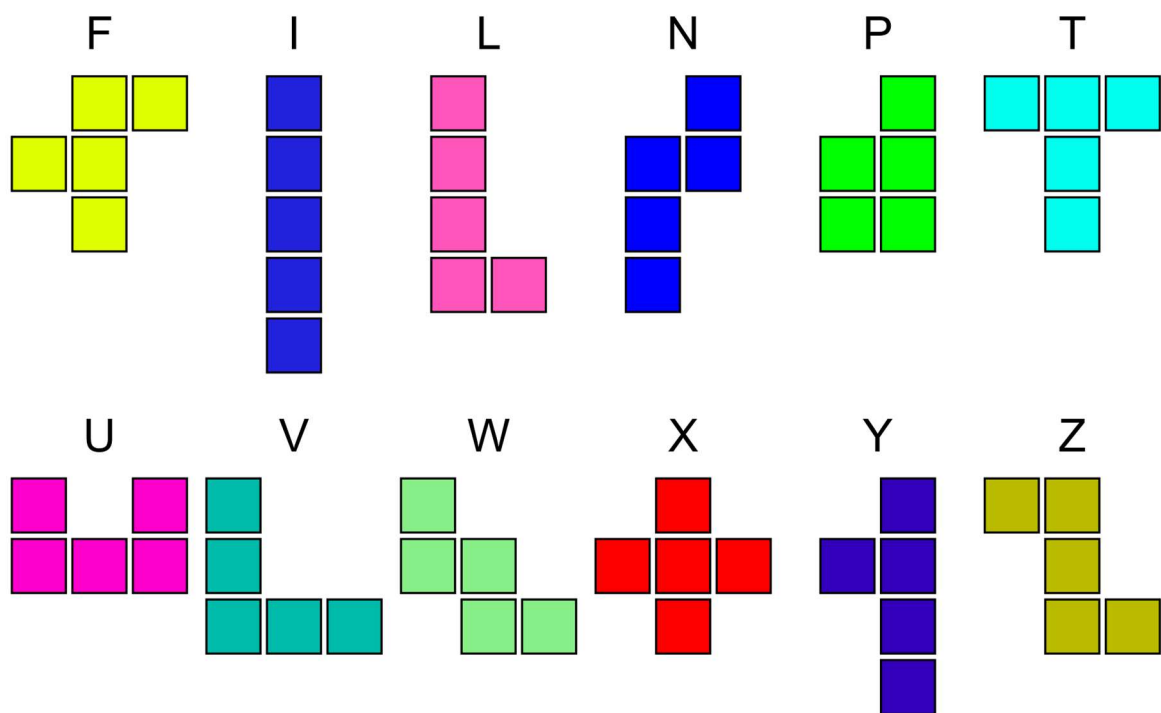


Figure 1 : Liste des Pentomino
<https://fr.wikipedia.org/wiki/Pentomino>

Toutes les images qui vont suivre sont des captures d'écran de l'affichage graphique de notre programme :

- Combinaisons possibles pour une grille de 5*3
 - LTY
 - LVN
 - PLV
 - UPF
 - UPN
 - UPY (représenté ci-dessous)

3	4	4	4	3
4	4	4	4	4
3	4	4	4	3

- Combinaisons possibles pour une grille de 5*4
 - PLTY
 - PLVY (représenté ci-dessous)
 - PLWY
 - ULFY
 - UPIV
 - UPIY

3	4	4	4	3
4	4	4	4	4
4	4	4	4	4
3	4	4	4	3

- Combinaisons possibles pour une grille de 5*5
 - IPYFU

- TLVWP
- LPVYN
- LTVWY
- PILTV
- PILTY
- PILVY
- PILVZ
- PILWY (représenté ci-contre)
- PILZY
- PITVW
- PLTWN
- PLTWY
- PLVFY
- PLVWF
- PLVYN
- PLVZY
- PLWZY
- PVWYN
- PVWZY
- TVFYN
- UILFY
- UILVF
- UILVY
- ULVZN
- ULWYN
- UPIFY
- UPILF
- UPILN
- UPILY
- UPIVZ
- UPIYN
- UPLTX
- UPLTZ
- UPLWF
- UPLZN

3	4	4	4	3
4	4	4	4	4
4	4	4	4	4
4	4	4	4	4
3	4	4	4	3

- UPTFY
- UPVFN
- UPVYN
- UPZYN
- Combinaisons possibles pour une grille de 5*11
 - PILTVWXZFYN
 - UILTVWXZFYN
 - UPILTVWXFYN
 - UPILTVWXZFN
 - UPILTVWXZFN
 - UPILTVWXZYN
 - UPILTVWZFYN
 - UPILTVXZFYN
 - UPILTWXZFYN
 - UPILVWXZFYN
 - UPITVWXZFYN
 - UPLTVWXZFYN
- Combinaisons possibles pour une grille de 5*12
 - UPILTVWXZFYN